

Digital Logic Design: 8-bit Adders Report

Ashkan Namjoo

June 19, 2026

1 Project Github Repo:

<https://github.com/rowyfol/ds1-project-2026/tree/main/Pj2>

2 Introduction

This report details the design and simulation of various digital components, culminating in the implementation of an 8-bit Ripple Carry Adder (RCA) and an 8-bit Carry Select Adder (CSA). All modules were described using Verilog with specific propagation delays to model realistic hardware behavior.

3 Module Designs

3.1 Full Adder (1-bit)

The foundation of our adders is the 1-bit Full Adder module (`full_adder`). It takes two bits A, B and a carry-in C_{in} as inputs. We assigned a delay of $t_{sum} = 10\text{ ns}$ for the sum output and $t_{co} = 8\text{ ns}$ for the carry-out (C_o). These values were chosen arbitrarily but reflect common hardware characteristics where carry logic is prioritized.

3.2 4-bit Ripple Carry Adder

We designed a 4-bit adder (`adder4`) by cascading four 1-bit Full Adders. The carry output of each bit is linked to the carry input of the next bit. This module serves as a sub-component for our larger 8-bit designs.

3.3 Multiplexers (1-bit and 4-bit)

A 2-to-1 multiplexer (`mux2_to_1`) was implemented with a propagation delay of $t_{mux} = 5\text{ ns}$. To handle multi-bit data, we created a 4-bit multiplexer (`mux4bit`) by instantiating four 1-bit MUXes in parallel.

3.4 8-bit Adders

Two different architectures were explored for 8-bit addition:

1. **8-bit Ripple Carry Adder (RCA):** This is a straightforward extension of the ripple carry principle, using 8 Full Adder modules in a single chain.
2. **8-bit Carry Select Adder (CSA):** This design uses one 4-bit adder for the lower bits and two separate 4-bit adders for the upper bits (pre-calculating results for both $C_{in} = 0$ and $C_{in} = 1$). A 4-bit MUX then selects the correct upper sum, and a 1-bit MUX selects the final C_{out} .

4 Simulation and Verification

Each module was thoroughly tested to ensure logical correctness. We used testbenches that applied various input combinations, specifically focusing on three key scenarios:

- **Scenario 1: Baseline Addition.** Adding small numbers (like $0x00 + 0x00$) to verify basic functionality.
- **Scenario 2: Carry Propagation.** Inputs like $0x0F + 0x01$ were used to ensure the carry correctly transitions from the lower nibble to the upper nibble.
- **Scenario 3: Maximum Load.** Adding $0xFF + 0xFF$ with an initial $C_{in} = 1$ to test the circuit under maximum carry-out conditions.

The simulation results were captured in VCD (Value Change Dump) files and verified using GTKWave, confirming that all outputs matched the expected theoretical values.

5 Delay Analysis and Comparison

A critical part of this project was analyzing how different architectures affect performance. We used the following delay parameters: $t_{sum} = 10$ ns, $t_{co} = 8$ ns, and $t_{mux} = 5$ ns.

Ripple Carry Adder (RCA): In an 8-bit RCA, the critical path is the carry propagation. The n -th sum bit is ready at $(n - 1)t_{co} + t_{sum}$. For 8 bits ($n = 8$):

$$\text{Delay}_{RCA} = 7 \times 8 \text{ ns} + 10 \text{ ns} = 66 \text{ ns}$$

The final carry-out is ready at $8 \times 8 \text{ ns} = 64 \text{ ns}$.

Carry Select Adder (CSA): The CSA speeds things up by pre-calculating the upper 4 bits. The delay is determined by the lower 4-bit carry-out plus the multiplexer selection time:

$$\text{Delay}_{CSA} = (4 \times 8 \text{ ns}) + 5 \text{ ns} = 37 \text{ ns (for } C_{out})$$

The upper sum bits are ready slightly later at $34 \text{ ns} + 5 \text{ ns} = 39 \text{ ns}$.

Final Verdict: The Carry Select Adder achieves a significant performance boost. By "spending" more hardware to pre-calculate potential outcomes, we reduced the total critical path delay from **66 ns** to **39 ns** (a 40% improvement).

This makes the CSA much more suitable for high-speed arithmetic units where performance is more critical than silicon area.

Metric	Ripple Carry Adder	Carry Select Adder
Critical Path (Worst Case)	66 ns	39 ns
Complexity	Low (8 FAs)	High (12 FAs + MUXes)
Performance	Standard	Optimized

Table 1: Comparison of 8-bit Adder Architectures

6 Summary of Files

All source files (`.v`) and testbenches are provided in their respective directories. The simulation waveforms for all 7 parts are included in the generated `.vcd` files, which can be viewed for detailed timing analysis.